

Estudiantes:

- Manuel Alexander Tziboy Catún - 26085

- Juan Fernando Santos Mérida - 26693

Curso: Programación Orientada a Objetos

Sección: 30

Laboratorio 2 - Análisis

Análisis:

Clase	Propiedades	Métodos
Main		• public static void main(String[] args): Es el módulo principal desde donde se llamará el inicializador.
Festival	• Private String nombre: almacena la variable del nombre del festival. • Private String código: almacena la variable del identificador del festival. • Private String coordinador: almacena la variable del nombre del responsable del festival. • Private Escenario[] escenarios: almacena la lista de los escenarios.	• public Festival(String nombre, String codigo, String coordinador): Es el constructor de la clase, le da sus valores iniciales. • public String getNombre(): devuelve el nombre del festival. • public String getCodigo(): devuelve el codigo del festival. • public String getCoordinador(): devuelve el nombre del coordinador del festival. • public Escenario[] getEscenarios(): devuelve la lista de escenarios del festival.

Escenario	• Private String codigo: almacena el identificador del escenario. • Private String nombre: guarda el nombre del escenario • Private String ubicacion: guarda la ubicación del escenario • Private int capacidad: almacena la capacidad máxima de asistentes en el escenario • Private String estado: indica el estado del escenario • Private ArrayList<Artista> artistas: guarda la lista de artistas del escenario	• Public Escenario(String código, String nombre, String ubicacion, int capacidadMaxima, String estado): Es el constructor de la clase, le da sus valores iniciales. • Public String getCodigo(): devuelve el código del escenario • Public String getNombre(): obtiene el nombre del escenario • Public String getUbicacion(): obtiene la ubicación del escenario • public int getCapacidad(): obtiene la capacidad de asistentes del escenario • public String getEstado(): obtiene el estado del escenario • public void setCapacidad(int nuevaCapacidad): Modifica la capacidad máxima de asistencia del escenario • public void setEstado(String nuevoEstado): modifica el estado del escenario • Public String toString(): devuelve la información completa del escenario a un String
Artista	• Private String codigo: almacena la variable del identificador del artista. • Private String nombreArtistico: almacena la variable del	• Public Artista(String codigo, String nombreArtistico, String generoMusical, int duracionPresentacion, int asistentesEstimados) • Public String getCodigo(): obtiene el código del artista • Public String getNombre(): obtiene el nombre del artista

	nombre artístico del artista. • Private String generoMusical: almacena la variable del genero musical del artista. • Private double duracion: almacena la variable de la duración del show en minutos. • Private int asistentes: almacena la variable del número estimado de asistentes.	• Public String getGenero(): obtiene el genero musical del artista • Public double getDuracion(): obtiene la duración de la presentación del artista • Public int getAsistentes(): obtiene la cantidad de asistentes de la presentación del artista • Public void setNombre(String nombreNuevo): modifica el nombre de un artista • Public void setGenero(String nuevoGenero): modifica el genero musical del artista • Public void setDuracion(double nuevaDuracion): modifica la duración de la presentación del artista • Public void setAsistentes(int cantidadAsistentes): modifica la cantidad de asistentes estimados • Public String toString(): devuelve la información en un string
ControladorFestival	• Private Festival festival: para almacenar los datos del festival. • Private VistaEvento vista: para llevar a cabo la interacción con el usuario.	• public ControladorFestival(Festival festival, VistaFestival vista): es el constructor de la clase, le da sus valores iniciales. • public void iniciar(): inicializa el programa y controla el menú principal. • private void ejecutarOpcion(int opcion): ejecuta la opción seleccionada por el usuario. • private void configurarEscenario(): crea y almacena un escenario en una posición disponible.

		• private void consultarEscenarios(): obtiene todos los escenarios configurados. • private void consultarEscenario(): obtiene el escenario de una posición específica. • private void modificarEscenario(): modifica la capacidad y el estado de un escenario. • private void retirarEscenario(): elimina un escenario y deja su posición en null. • private void registrarArtista(): registra un artista en el escenario seleccionado. • private void consultarArtistas(): obtiene todos los artistas registrados. • private void buscarArtista(): busca un artista por medio de su código. • private void modificarArtista(): modifica la información de un artista registrado. • private void cancelarParticipacion(): elimina un artista del escenario donde está registrado. • private void mostrarReporte(): calcula y muestra los datos generales del festival. • private boolean posicionValida(int posicion): verifica si una posición pertenece al arreglo.
--	--	---

		• private Artista buscarArtistaPorCodigo(String codigo): busca un artista en todos los escenarios. • private Escenario buscarEscenarioDelArtista(String codigo): devuelve el escenario donde está registrado un artista. • private int contarEscenariosConfigurados(): devuelve la cantidad de escenarios configurados. • private int contarEspaciosDisponibles(): devuelve la cantidad de posiciones disponibles. • private Escenario obtenerEscenarioMayorCapacidad(): devuelve el escenario con mayor capacidad. • private int contarArtistas(): devuelve la cantidad total de artistas registrados. • private Artista obtenerArtistaMayorDuracion(): devuelve el artista con la presentación más larga. • private Artista obtenerArtistaMayorAsistencia(): devuelve el artista con más asistentes estimados. • private double calcularPromedioDuracion(): devuelve el promedio de duración de las presentaciones.
VistaFestival	• Private Scanner sc: Para leer los datos que ingrese el usuario.	• public VistaFestival(): es el constructor de la clase e inicializa el Scanner.

		• <code>public int mostrarMenu():</code> muestra las opciones y devuelve la opción seleccionada. • <code>public void</code> <code>mostrarMensaje(String mensaje):</code> muestra un mensaje al usuario. • <code>public void mostrarError(String mensaje):</code> muestra un mensaje de error. • <code>public void</code> <code>mostrarFinOperacion():</code> muestra un mensaje al finalizar una operación. • <code>public void limpiarScanner():</code> limpia una entrada incorrecta del Scanner. • <code>public void cerrarScanner():</code> cierra el Scanner al finalizar el programa.
--	--	--

Preguntas

3. La propiedad que debe implementarse utilizando un arreglo básico es la colección de escenarios del festival. Esta propiedad puede declararse dentro de la clase Festival como un arreglo de objetos de tipo Escenario: `escenarios = new Escenario[5];`
El arreglo tendrá un tamaño fijo de 5 posiciones, debido a que cada festival puede tener como máximo cinco escenarios. Las posiciones que aún no tengan un escenario configurado deberán contener null
4. La colección de artistas participantes debe implementarse utilizando un ArrayList, debido a que la cantidad de artistas puede cambiar durante la organización del festival: `private List<Artista> artistas = new ArrayList<>();`
7. Los objetos recibirán sus valores iniciales mediante los constructores. En Escenario, la capacidad máxima debe ser mayor que 0. En Artista, la duración debe ser mayor que 0 y la cantidad estimada de asistentes debe ser mayor o igual que 0. Si estas condiciones no se cumplen, se debe generar una `IllegalArgumentException`.

8. Para determinar si una posición contiene un escenario, primero se debe comprobar que el índice se encuentre dentro de los límites del arreglo y posteriormente comparar el elemento de esa posición con null.
9. Las operaciones se realizarán recorriendo el `ArrayList<Artista>` y utilizando el código del artista como criterio para localizarlo. Para la búsqueda, se recorrerá el `ArrayList` con un `For-Each` hasta encontrar un artista cuyo código coincida con el código solicitado.
10. El programa puede producir principalmente dos tipos de excepciones requeridas explícitamente por el laboratorio: `InputMismatchException` e `IllegalArgumentException`. La `InputMismatchException` puede ocurrir cuando el usuario introduce mediante `Scanner` un dato cuyo tipo no corresponde con el solicitado. Por ejemplo, escribir "cinco" cuando el programa espera un `int`. Esta excepción deberá manejarse principalmente en la interacción con el usuario, por lo que puede utilizarse `try-catch` desde la clase `Vista` o donde se realicen las lecturas con `Scanner`. La `IllegalArgumentException` se producirá cuando se intente crear o modificar un objeto utilizando valores que incumplan las reglas del sistema. Por ejemplo:
 - a. `capacidadMaxima <= 0` en un `Escenario`
 - b. `duracionPresentacion <= 0` en un `Artista`
 - c. `asistentesEstimados < 0` en un `Artista`